for $\forall \varphi(x,y,t) \in V_h^k(t)$,

$$\begin{aligned}
&\frac{d}{dt}\iint_{I_{ij}(t)} (\mathbf{U}\varphi(x,y,t))\,dxdy \\
=&\iint_{I_{ij}(t)} \left(\mathbf{U}\left(\mathbf{u}\cdot\nabla\varphi + \frac{\partial\varphi}{\partial t}\right)\right)dxdy - \int_{\partial I_{ij}^{Ncs}(t)} \left(\widehat{\mathbf{U}}(\widehat{\mathbf{u}}-\mathbf{u}_g)\cdot\mathbf{n}\varphi\right)ds \\
&+\iint_{I_{ij}(t)} (\mathbf{G}\cdot\nabla\varphi)\,dxdy - \int_{\partial I_{ij}(t)} \left(\widehat{\mathbf{G}}\cdot\mathbf{n}\varphi\right)ds \\
&-\sum_{a=0}^{k}\frac{\sigma_{ij}^a}{h_{ij}}\iint_{I_{ij}(t)} \left(\mathbf{U}-P_h^{a-1}\mathbf{U}\right)\varphi dxdy,
\end{aligned} \tag{3.1}$$

where $\partial I_{ij}^{Ncs} = \partial I_{ij}\backslash\Gamma_{cs}^{ij}$ when $\varphi = 1$, otherwise $\partial I_{ij}^{Ncs} = \partial I_{ij}$, see the DG scheme (2.9)-(2.10) for details. Here we call $-\sum_{a=0}^{k}\frac{\sigma_{ij}^a}{h_{ij}}\iint_{I_{ij}(t)}\left(\mathbf{U}-P_h^{a-1}\mathbf{U}\right)\varphi dxdy$ the damping term. h_{ij} is the longest edge length of the cell I_{ij}. P_h^a is the standard L^2 projection into $\mathbf{V}_h^a$, $a \geq 0$, i.e., for any vector function $\mathbf{w}$, $P_h^a\mathbf{w} \in \mathbf{V}_h^a$ satisfies

$$\iint_{I_{ij}} (P_h^a\mathbf{w}-\mathbf{w})\varphi dxdy = 0,\ \forall\varphi \in p^a(I_{ij}), \tag{3.2}$$

and we set $P_h^{-1} = P_h^0$. By (3.2), we can find that $\iint_{I_{ij}(t)}\left(\mathbf{U}-P_h^{a-1}\mathbf{U}\right)dxdy = 0,\ a = 0,1,\cdots,k$, that is, the damping term does not play a role in the cell average equations, so the conservation of the above scheme is not lost.

One key aspect of controlling numerical oscillations is the design of the damping coefficients σ_{ij}^a. These coefficients σ_{ij}^a are non-negative, small in the smooth regions and large near the discontinuities. Referring to [29], the damping coefficients are given as follows:

In my view, it is $\frac{dU}{dt} = -\lambda\left(U - P^{a-1}U\right)$ | Motivation

if $U = \begin{cases} U_{low} = P^{a-1}U \\ + \\ U_{high} = U - P^{a-1}U \end{cases} \Rightarrow \frac{\partial U_{low}}{\partial t} + \frac{\partial U_{high}}{\partial t} = -\lambda U_{high}$

$\Rightarrow$ (not trivial but works) $\begin{cases} \frac{\partial U_{low}}{\partial t} = 0 \\ \frac{\partial U_{high}}{\partial t} = -\lambda U_{high} \end{cases}$

Let's play analysis

Ω : polynomial function space with $\langle f, g \rangle = \int fg$

norm L_2

$$U = PU + V$$

$$\|U\|_{L_2}^2 = \|PU + V\|_{L_2}^2 = \int (PU+V)(PU+V)$$

$$= \|PU\|_{L_2}^2 + \|V\|_{L_2}^2$$

$$\text{So } \|PU\|_{L_2}^2 \le \|U\|_{L_2}^2$$

P project is continuous operator

By linearity

$$P\left(\frac{U(t+\delta t) - U(t)}{\delta t}\right) = \frac{PU(t+\delta t) - PU(t)}{\delta t}$$

$$\lim_{\delta t \to 0} P(\quad) = \frac{\partial PU}{\partial t}$$

by P is continuous

$$P\left(\lim_{\delta t \to 0}(\quad)\right) \quad P\frac{\partial U}{\partial t} = \frac{\partial PU}{\partial t}$$

$$\text{So } \frac{dU_{low}}{\partial t} = P\frac{dU}{\partial t} = P(-\lambda(I-P)U)$$

$$= -\lambda(PI - P^2)U$$

by $P^2 = P$ $= 0$

$$\left(\frac{dU_{high}}{\partial t} = (I-P)\frac{dU}{\partial t} = -\lambda(I-P)(I-P)U\right.$$

$$= -\lambda(I - P - P + P^2)U$$

$$\left. = -\lambda(I-P)U = -\lambda U_{high}\right)$$

Which tells that

$$U_{high}(t) = e^{-\lambda t} U_{high}(0)$$

$$U_{low}(t) = U_{low}(0)$$

Again $\frac{dU}{dt} = -\lambda (I-P) U$

weak form $\left\langle \frac{dU}{dt}, \varphi \right\rangle = \left\langle -\lambda (I-P) U, \varphi \right\rangle$

$$\int \left(\frac{dU}{dt} + \lambda (I-P) U \right) \varphi = 0$$

Let's write the code

1st we need decide the damping coefficients σ_{ij}^{a}

One key aspect of controlling numerical oscillations is the design of the damping coefficients σ_{ij}^a. These coefficients σ_{ij}^a are non-negative, small in the smooth regions and large near the discontinuities. Referring to [29], the damping coefficients are given as follows:

$$\sigma_{ij}^a = \frac{2(2a+1)}{(2k-1)} \frac{\widetilde{h}_{ij}^a}{a!} \max_{1\le q\le 4} \sum_{|\boldsymbol{\alpha}|=a} \left(\frac{1}{N_e} \sum_{\boldsymbol{v}\in\partial I_{ij}} \left([\![\partial^{\boldsymbol{\alpha}} V_q]\!]\big|_{\boldsymbol{v}} \right)^2 \right)^{\frac{1}{2}}, \tag{3.3}$$

where $\widetilde{h}_{ij} = \max\{h_{ij}, h_{i\pm1,j}, h_{i,j\pm1}\}$. The vector $\boldsymbol{\alpha} = (\alpha_1, \alpha_2)^T$ is the multi-index with $|\boldsymbol{\alpha}| = \alpha_1 + \alpha_2$ in two dimensions, and $\partial^{\boldsymbol{\alpha}} w$ is defined as

$$\partial^{\boldsymbol{\alpha}} w = \frac{\partial^{|\boldsymbol{\alpha}|} w}{\partial x^{\alpha_1} \partial y^{\alpha_2}}.$$

$\boldsymbol{v} = (x_{i\pm\frac{1}{2},j\pm\frac{1}{2}}, y_{i\pm\frac{1}{2},j\pm\frac{1}{2}})^T$ is the vertex of the cell I_{ij}. $[\![w]\!]\big|_{\boldsymbol{v}}$ represents the jump of the function w at the vertex $\boldsymbol{v}$ of two neighboring cells that share the same edge. N_e is the number of edges of the cell I_{ij}, and we take $N_e = 4$ in the quadrilateral mesh.

Given degree-k → Case_option → Discretization

fix Cell i,j

$\alpha \in \{0, 1, \dots, k\}$

function $G(i,j)$ $\begin{bmatrix} G^0 \\ \vdots \\ G^k \end{bmatrix}_{k+1}$ → Sigma_mat (cell_index)

$\frac{2\cdot(2\alpha+1)}{2k-1} \cdot \widetilde{h}_{ij}^{\alpha} / \alpha!$

$\widetilde{h}_{ij}$ longest edge all over 5 cell (itself and all adject 4cell)

We don't save neighbor index for each cell...

But we can use face_list

Assume $\tilde{h_{ij}} = h_{ij}$ ⟶ [geometry] → Cells_geometries

for face in faces

$$\tilde{h}[left] = \max\left(\tilde{h}[left], h[right]\right)$$

$$\tilde{h}[right] = \max\left(\tilde{h}[right], h[left]\right)$$

Done $\tilde{h_{ij}}$

Then we want to get

$$\max_{q=1,\cdots,4} \sum_{|\alpha|=\alpha} \left(\frac{1}{4} \sum_{v \in \partial\tau_{ij}} \left[\!\left[\partial^\alpha V_q \right]\!\right]^2 \right)^{\frac{1}{2}}$$

$q=1\cdots4$ means $V = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$ has 4 component

$$\partial^\alpha V_q = \left(R^{-1}\right)\Big|_{n_L} \partial^\alpha U\Big|_v$$

we do it at Euler.jl

left_eigen_vector(ave_cons, gamma, normal)

So we need compute jump of $\partial^\alpha U$ for all $|\alpha|$

Still we need loop based on face to calculate traces for $\partial^\alpha U$ $\forall \alpha$ at to vex of each face

① geo.eval_cons(~)

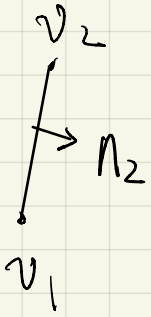
② for ghost cell I put it in Faceflux

And design at here (end of the file)

function cells_jump_V(dx, dy) $\left(\sum_{v \in \partial_{ij}} \| \partial^{dx dy} V_q \|^2 \frac{1}{4}\right)^{\frac{1}{2}}$

$$jump_V = \underbrace{[\quad 0 \quad]}_{\#\,cell}\,]4$$

for each face



we get

$$Jump_dx_dy_U = \left(\partial x^{dx}\partial y^{dy} U_{Left}^{(v_i)} - \partial x^{dx}\partial y^{dy} U_{right}^{(v_i)}\right) = 2\begin{bmatrix} J_1 \\ J_2 \\ J_3 \\ J_4 \end{bmatrix}$$

$$jump_V[:, face1_left_cell] \mathrel{+}= \left(R^{-1}_{Left}\, Jump_dx_dy_U\right)$$

$$jump_V[:, face1_right_cell] \mathrel{+}= \left(R^{-1}_{right}\, Jump_dx_dy_U\right)^2$$

return $\left(\frac{1}{4} jump_V\right)^{\frac{1}{2}}$ which is

$$\left[\frac{1}{4}\sum_{v \in \partial_{ij}} \left(R^{-1}\Big|_{\vec{n}}\, \partial x^{dx}\partial y^{dy}\, U\Big|_{v}\right)^2\right]^{\frac{1}{2}}$$

$$\begin{bmatrix} \left(\vdots\right) \end{bmatrix}$$

Given $|\alpha| = a$ fix a cell (i,j)

$$\max_{q=1,\dots,4} \quad \text{we want} \quad \sum_{|\alpha|=a} \left(\sum_{\text{vert}_{ij}} \| \partial^\alpha V_q \|^2 \frac{1}{4} \right)^{\frac{1}{2}}$$

jump_mat = $\Big[\underbrace{[\ 0\ ]}_{\#\text{cell}} \Big] 4$

fix $|\alpha| = a$ $\quad dx + dy = a \quad dx = 0, \dots, a$ $\quad dy = a - 0 = a - dx$

get $-(dx, dy)$ as derivatives

(fix a) for (dx, dy) in derivatives

jump_mat += cells_jump_V (dx, dy)

max_jump = max (for each cd, jump_mat [----]

$$\left[\sigma_{ij}^{a} \quad - \quad - \quad - \quad \sigma_{ij}^{a} \right]$$

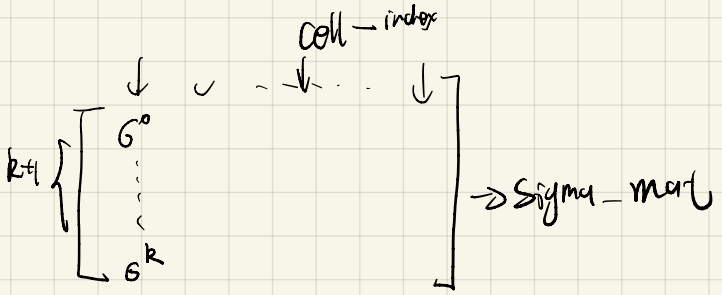
Sigma $\left[a_{a+1}, i \right]$ (in jth)

$$= \frac{2 \cdot (2a+1)}{2k-1} \cdot \tilde{h}_{ij}^{a} / a! \left[\text{max_jump} \right]$$

↑ loop for all $a = 0, \cdots, k$

we got

$\sigma(i,j)$



Done!

Shu didn't claim the how to deal with those ghost so I make my choose here

I move the design of this function here

But put code in Faceflux

outside_cons_derivative(face, pt_coord, normal
cells_coeff, cells_geometry,
problem,
exponents_ls
dx, dy)

Case ① dx+dy>0 back to outside_cons

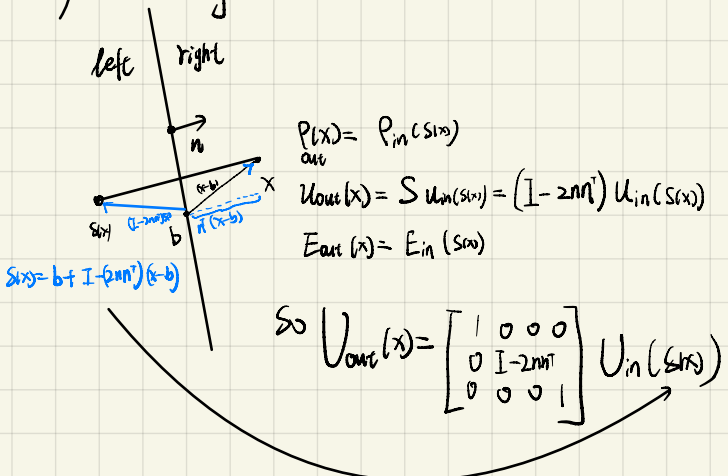
Now dx+dy>0

case boundary ("fix") return $\vec{0}$

("outflow") eval_cons using left cell $\partial^\alpha U_{out}(v) = \partial^\alpha U_{in}(v)$

"periodic" would be here

only "wall" (reflection) ghost state



So we can use chain rule reduce it as derivative of U_{in}

$$\partial_{x_1} V_{out}(\vec{x}) = \begin{bmatrix} \partial_{x_1} V_{out,1}(\vec{x}) \\ \vdots \\ \partial_{x_1} V_{out,l}(\vec{x}) \end{bmatrix} = \partial_{x_1}\left((1,0,\dots,0)^t \cdot V_{in}(S(\vec{x}))\right)$$

$$\partial_{x_1}\left((0\cdots 01)^t V_{in}(S(\vec{x}))\right)$$

$$\partial_x T \cdot V_{in}(S(\vec{x})) = \begin{pmatrix} 0 & \cdots & 0 \\ \vdots & & \\ 0 & \cdots & 1 \end{pmatrix} \partial_x \left(V_{in}(S(\vec{x}))\right)$$

$$= T\, \partial_x\left(V_{in}(S(\vec{x}))\right)$$

$$\partial_{x_1} V_{in}(S(x,y)) = \partial_1 V_{in}(S(x)) \cdot \partial_{x_1} s_1 + \partial_2 V_{in}\, \partial_{x_1} s_2$$

$$= \begin{bmatrix} \partial_1 V[1] & \partial_2 V[1] \\ \vdots & \\ \partial_1 V[\varphi] & \partial_2 V(\varphi) \end{bmatrix} \Bigg|_{\text{at } (S(\vec{x}))} \begin{pmatrix} \partial_{x_1} s_1 \\ \partial_{x_n} s_n \end{pmatrix}$$

$$= DU\Big|_{\text{at } S(x)} \cdot D_{x_i} S(\vec{x})$$

$$\partial_{x_i} U_{out}(\vec{x}) = T \cdot DU\Big|_{\text{at } S(x)} D_{x_i} S(\vec{x})$$

$$S(x) = b + (I - 2nn^T)(x - b)$$

$$\partial_{x_i} S(x) = \left[I - 2nn^T\right] \partial_{x_i} \begin{pmatrix} x_1 - b_1 \\ \vdots \\ x_n - b_n \end{pmatrix}$$

$$= \left[I - 2nn^T\right] \begin{pmatrix} 0 \\ \vdots \\ 1 \leftarrow i\text{th} \\ \vdots \\ 0 \end{pmatrix}$$

reflect_M

which only depend on $\vec{n}$ since $\vec{n}$ is fixed

$\partial_{x_i} S(x)$ is fixed for all x_i

As for high order derivative

$$\partial_{x_i}^{d_x} \partial_{y_i}^{d_y} U_{out}(\vec{x})$$

$$= T \cdot \partial_{x_i}^{d_x} \partial_{y_i}^{d_y} U_{in}(S(x)) \quad \cdots (*)$$

$$\begin{bmatrix} M_1 & M_2 \\ M_3 & M_4 \end{bmatrix} \begin{bmatrix} x_1 - b_1 \\ x_2 - b_2 \end{bmatrix}$$

$S_1 = M_1(x_1 - b_1) + M_2(x_2 - b_2)$

$\partial_{x_1} S_1 = M_1 \quad \partial_{x_2} S_1 = M_2$

$\partial_{x_1} S_2 = M_3$

$\partial_{x_2} S_2 = M_4$

$\forall (x,y)$ Since $S = \begin{pmatrix} S_1(x,y) \\ S_2(x,y) \end{pmatrix}$ $\partial_{x_i} S_1 = S_{1i} = M[1\,i]$ $\partial_{y_i} S_1 = S_{12} = M[1\,2]$ $\partial_{x_i} S_2 = S_{2i} = M[2\,1]$ $\partial_{y_i} S_2 = S_{2i} = M[2\,2]$

IS Constant

For each component of $U_{in} = \begin{bmatrix} P_{in} \\ P v_{x_{in}} \\ P v_{y_{in}} \\ E_{in} \end{bmatrix} S(\vec{x})$

$$\partial_{x_i}^{d_x} P_{in}(S(\vec{x})) = \partial_{x_i}^{d_x - 1} (D_1 P_{in} \partial_{x_i} S_1 + D_2 P_{in} \partial_{x_i} S_2)$$

$$= \partial_{x_i}^{d_x - 1} (\partial_{x_i} S_1 D_1 + \partial_{x_i} S_2 D_2) P_{in}$$

$$= \partial_{x_i}^{d_x - 1} (S_{1i} D_1 + S_{2i} D_2) P_{in}$$

$$\partial_{x_i}(S_{1i} D_1 P_{in}) = S_{1i}\, \partial_{x_i} D_1 P_{in}(S(x)) = S_{1i} (S_{1i} D_1 D_1 P_{in} + S_{2i} D_2 D_1 P_{in}) = S_{1i}^2 D_{11} P_{in} + S_{1i} S_{2i} D_{21} P_{in}$$

$$\partial_{x_i}(S_{2i} D_2 P_{in}) = S_{2i}\, \partial_{x_i} D_2 P_{in}(S(x)) = S_{2i} S_{1i} D_{12} P_{in} + S_{2i}^2 D_{22} P_{in} \quad > S_{1i}^2 D_{11} P_{in} + 2 S_{1i} S_{2i} D_{12} P_{in} + S_{2i}^2 D_{22}$$

So $(*) = T \cdot (S_{12} D_1 + S_{22} D_2)^{d_y} (S_{1i} D_1 + S_{2i} D_2)^{d_x} U_{in}(S(\vec{x}))$

By $(a+b)^n = \sum_{i=0}^{n} C_n^i a^i b^{n-i}$

$$= T\cdot(S_{11}D_x + S_{21}D_y)^{dx}\ (S_{12}D_x + S_{22}D_y)^{dy}\ U_{in}(\vec{s})$$

$$= T\cdot\sum_{i=0}^{dx} C_{dx}^{i}\,(S_{11}D_x)^{i}(S_{21}D_y)^{dx-i}\sum_{j=0}^{dy} C_{dy}^{j}(S_{12}D_x)^{j}(S_{22}D_y)^{dy-j}\ U_{in}(\vec{s})$$

$$= T\cdot\sum_{i=0}^{dx}\sum_{j=0}^{dy} C_{dx}^{i} C_{dy}^{j} S_{11}^{i} S_{21}^{dx-i} S_{12}^{j} S_{22}^{dy-j} D_x^{i+j} D_y^{dx+dy-(i+j)}\ U_{in}(\vec{s})$$

geo.eval(cell_coeff, geomcoy, pt_coord, exponents, dx=i+j, dy=dx+dy-(i+j))

So we need (face, pt_coord, normal, cells_coeff, cells_geometry, problem, exponents_ls, dx, dy)

This function help us decide the jump between boundary cell and ghost cell.

[I don't know how Shu did it, maybe we can also skip it...]

For now, we have got σ_{ij}^{a}

Then we just need do filter by 3.5->3.8

Instead of put this term together to solve (3.1)

This filter will be done "analytically"

$$By\ U = C^{(0)}(t)\phi_0 + \sum_{m=1}^{K} C^{(m)}(t)\phi_m$$

(ϕ_0: fixed)

First we want to get $U - P^r U$

$\longrightarrow S = \{j : \deg \phi_j \le r\}$

By definition of projection $P^r U = \sum_{j=0}^{S} d_j \phi_j$

$$\langle U - P^r U, \phi_i \rangle = 0 \quad \forall i = 1, \dots, S$$

$$\iint \left[-\sum_{j=0}^{S} d_j \phi_j \phi_i + \sum_{j=0}^{K} C_j \phi_j \phi_i \right] = 0 \quad \forall i = 1, \dots, S$$

Recall $M_{ij} = \iint \phi_i \phi_j$

$$\sum_{j=0}^{K} C_j M_{ij} = \sum_{j=0}^{S} d_j M_{ij} \quad \forall i = 1, \dots, S$$

$$\begin{bmatrix} M_{11} & \cdots & M_{1K} \\ & & \\ M_{S1} & \cdots & M_{SK} \end{bmatrix} \begin{bmatrix} C_1 \\ \vdots \\ C_K \end{bmatrix} = \begin{bmatrix} M_{11} & \cdots & M_{1S} \\ & & \\ M_{S1} & \cdots & M_{SS} \end{bmatrix} \begin{bmatrix} d_1 \\ \vdots \\ d_S \end{bmatrix}$$

$\downarrow M_{S\times S}$

$$\begin{bmatrix} d_1 \\ \vdots \\ d_S \end{bmatrix} = M_{S\times S}^{-1} M_{S\times K} \vec{C}$$

function project_mat (Mass mat, project_degree, exps_ls)

indice = ∀ a+b ≤ project degree (a,b) in exp_ls

(k)

project_mat = zeros(k, k)

return m[:s, :] = M[:s, :] \ M[s:, :] (· ⃗) = ⃗

cell_coff = [⃗ ⃗ ⃗ ⃗]

Recall (3.5)

$$\begin{cases} \dfrac{d}{d\tilde{t}} \iint_{I_{ij}^{n,s+1}} (\mathbf{U}_\sigma \varphi)\, dxdy + \sum_{a=0}^{k} \dfrac{\sigma_{ij}^a}{h_{ij}} \iint_{I_{ij}^{n,s+1}} \left(\mathbf{U}_\sigma - P_h^{a-1}\mathbf{U}_\sigma\right) \varphi dxdy = 0, \\ \mathbf{U}_\sigma(x, y, 0) = \mathbf{U}^{n,s+1}(x, y), \quad \text{initial value} \quad \tilde{t} \ \text{How long?} \end{cases} \tag{3.5}$$

fix cell(ij)

$$\iint \left[\dot{U}_6 + \sum_{a=0}^{k} \frac{\sigma^a}{h} (I - P^{a-1}) U_6\right] \varphi \, dxdy = 0$$

$\|$ $\forall \varphi_i$ for each $v_i \in [U_1 \cdots U_4]$ $v_i = \Sigma c_j \varphi_j$

$$\sum_j \iint \dot{c}_j \varphi_j \varphi_i = -\sum_{a=0}^{k} \frac{\sigma^a}{h} \iint \underbrace{(I - P^{a-1}) \begin{bmatrix} c_0 \\ \vdots \\ c_k \end{bmatrix}}_{= \vec{r}} \vec{\varphi} \cdot \varphi_i$$

$$\sum_{j=0}^{k} \dot{c}_j \iint \varphi_i \varphi_j = -\sum_{a=0}^{k} \frac{\sigma^a}{h} \sum_{j=0}^{k} \left(\iint \varphi_j \varphi_i\right) b_j$$

$$\sum_j \dot{\vec{c}}_j M_{ji} = -\sum_{a=0}^{k} \sum_{j=0}^{k} \frac{\sigma^a}{h} b_j M_{ij} \quad \forall i$$

$$\begin{bmatrix} M \end{bmatrix} \begin{bmatrix} \dot{c}_0 \\ \vdots \\ \dot{c}_k \end{bmatrix} = -\sum_{a=0}^{k} \frac{\sigma^a}{h} \begin{bmatrix} M \end{bmatrix} \begin{bmatrix} b_1 \\ \vdots \\ b_M \end{bmatrix} = -\sum_{a=0}^{k} \frac{\sigma^a}{h} M (I - P^{a-1}) \begin{bmatrix} c_0 \\ \vdots \\ c_k \end{bmatrix}$$

So

$$\frac{d}{d\tilde{t}}\begin{pmatrix} C_0 \\ \vdots \\ C_N \end{pmatrix} = -\underbrace{M^{-1}\sum_{\alpha=0}^{k}\frac{\sigma_\alpha}{h}M\left(I-P^{\alpha-1}\right)}_{B}\vec{C}$$

↳ as damping matrix

which is B in (3.9)

$$C(\Delta\tilde{t}) = e^{-(\Delta\tilde{t}B)}C(0)$$

Done.